



ADMINISTRACIÓN DE SERVIDORES

DOCUMENTACIÓN SERVIDOR WEB

PROYECTO GRUPAL

José Luis Olmo Barberá
Victor Manuel Vidal Molina
Alvaro Griñón Martínez

CURSO 2024/25

Índice

1. Explicación contenido añadido a la infraestructura	2
1.1. Contenedor Apache-Web en Production	3
1.1.1. Código del contenedor apache_web en el docker-compose.yml . . .	3
1.1.2. Especificaciones técnicas	3
1.1.3. Código del dockerfile usado para crear una imagen personalizada . .	4
1.1.4. Código del script 'entrypoint.sh' usado en el Dockerfile	5
1.1.5. Explicación dockerfile y entrypoint	6
1.1.6. Código del archivo 'default.conf'	7
1.1.7. Explicación contenido 'default.conf'	7
1.1.8. Comprobaciones dentro del contenedor:	9
1.1.9. Acceso a Apache desde el host:	10

1. Explicación contenido añadido a la infraestructura

Durante esta fase de implementación, se han incorporado un nuevo contenedor a la infraestructura anterior. A continuación se detallan las características y configuraciones implementadas en el contenedor en el área production:

■ Servidor Apache-Web:

Los requisitos del contenedor son los siguientes:

- Servidor apache http público, con un html en blanco con título y el nombre de los participantes en el cuerpo.
- Se requiere dar acceso por http a los contenidos de la carpeta "publico" del servidor FTP montado en la Práctica 2. Además, este acceso estará restringido mediante login.
- Será necesario colocar un enlace a de acceso a FTP en la página principal.
- El servidor web abrirá una página con un error específico ante cualquier petición que reciba a un archivo con extensión php.
- No se tiene previsto utilizar el protocolo https.

1.1. Contenedor Apache-Web en Production

1.1.1 Código del contenedor apache_web en el docker-compose.yml

```
# Bloque servidor WEB en production
apache_web:
  build: ./production/apache_web
  container_name: apache_web
  restart: always
  cap_add:
    - NET_ADMIN
  volumes:
    - ./development/samba_ftp/data/publico:/mnt/publico:ro
  ports:
    - "8082:80"
  dns:
    - 172.20.0.6
  networks:
    bridge_local: {}
    production_net:
      ipv4_address: 172.30.0.20
```

Figura 1: Contenedor apache_web.

1.1.2 Especificaciones técnicas

- Nombre del contenedor: apache_web.
- Reinicio automático si se detiene por error.
- NET_ADMIN otorga privilegios administrativos de red al contenedor, necesario para ejecutar el contenido del entrypoint correctamente.
- IP asignada: 172.30.0.20.
- DNS: 172.20.0.6.
- Puertos: '8082:80'.
- Redes: production_net que lo conecta a la infraestructura y bridge_local que conecta el host con el contenedor.

1.1.3 Código del dockerfile usado para crear una imagen personalizada

```
# apache_web /Dockerfile
FROM debian:bullseye

RUN apt-get update && apt-get install -y \
    sudo \
    nano \
    net-tools \
    iputils-ping \
    dnsutils \
    iproute2 \
    curl \
    netcat-openbsd \
    apache2 \
    apache2-utils \
    && rm -rf /var/lib/apt/lists/*

RUN a2enmod headers auth_basic

RUN useradd -m john && echo "john:john" | chpasswd && adduser john sudo

COPY errorphp.html /var/www/html/errorphp.html
COPY index.html /var/www/html/index.html
COPY default.conf /etc/apache2/sites-available/000-default.conf
COPY entrypoint.sh /entrypoint.sh

RUN chmod +x /entrypoint.sh

EXPOSE 80

ENTRYPOINT ["/entrypoint.sh"]
```

Figura 2: Dockerfile para crear imagen personalizada de apache_web.

1.1.4 Código del script 'entrypoint.sh' usado en el Dockerfile

```
#!/bin/bash

# Configurar gateway para producción
ip route del default 2>/dev/null
iface_prod=$(ip -o -4 addr show | grep 172\.30\. | awk '{print $2}' | head -n1)
if [[ -n "$iface_prod" ]]; then
    echo "Asignando gateway 172.30.0.1 a través de $iface_prod"
    ip route add default via 172.30.0.1 dev "$iface_prod"
fi

# DNS interno
echo "nameserver 172.20.0.6" > /etc/resolv.conf

htpasswd -cb /etc/apache2/.htpasswd documentos GraTiS!

exec apache2ctl -D FOREGROUND
```

Figura 3: Contenido script ejecutado en el dockerfile de apache_web.

1.1.5 Explicación dockerfile y entrypoint

■ Dockerfile:

- Actualizamos e instalamos los paquetes necesarios en el contenedor.
- Habilitamos los módulos necesarios de Apache.
- Creamos el usuario John y damos permisos, que aunque no se especificaba nada, al igual que en la práctica anterior lo hemos añadido a todos los contenedores.
- Copiamos los archivos necesarios desde fuera del contenedor hacia dentro y otorgamos permisos de ejecución al script 'entrypoint.sh'.
- Exponemos el puerto 80.
- Por último, ejecutamos el script 'entrypoint.sh' como comando principal del contenedor.

■ Entrypoint:

- Eliminamos la ruta por defecto.
- Detectamos la interfaz de production_net y la establecemos como gateway.
- Establecemos el servidor DNS.
- Creamos el archivo '.htpasswd' con el usuario indicado en el guión y el hash de la contraseña indicada, este archivo es necesario para poder verificar las credenciales a la hora de hacer login de acceso al FTP desde la página web.
- Finalmente, iniciamos el servidor apache en primer plano.

1.1.6 Código del archivo 'default.conf'

```
<VirtualHost *:80>
    DocumentRoot /var/www/html

    Alias /documentacion/ "/mnt/publico/"
    <Directory "/mnt/publico/">
        Options Indexes FollowSymLinks
        AllowOverride AuthConfig
        AuthType Basic
        AuthName "Área restringida"
        AuthUserFile "/etc/apache2/.htpasswd"
        Require user documentos
    </Directory>

    <FilesMatch "\.php$">
        ForceType text/plain
        Header set Content-Type "text/html"
    </FilesMatch>

    ErrorDocument 404 /errorphp.html
</VirtualHost>
```

Figura 4: Contenido del archivo 'default.conf'.

1.1.7 Explicación contenido 'default.conf'

A continuación vamos a explicar cada uno de los bloques línea por línea:

- Indicamos que esta configuración se aplicará a cualquier dirección IP (*) que reciba una petición en el puerto 80 (HTTP).
- Define el directorio raíz del sitio web.
- Crea un alias de URL: cualquier petición a /documentacion/ se servirá con archivos desde /mnt/publico/ .
- Abre un bloque de configuración específico para el directorio /mnt/publico/, donde se encuentra la documentación protegida.

- Permitimos opciones en el directorio como mostrar un listado de archivos si no hay index.html y seguir enlaces simbólicos dentro del directorio.
- Permitimos que los archivos .htaccess puedan sobrescribir la configuración de autenticación.
- Indicamos que se utilizará autenticación básica.
- "Área Restringida" será el texto que se mostrará en el cuadro de diálogo del navegador al pedir las credenciales.
- Indicamos la ruta del archivo .htpasswd que contiene los usuarios y hashes de contraseñas autorizados para acceder a este directorio.
- Solo se permite el acceso al usuario llamado documentos.
- Cerramos el bloque <Directory>. Todo lo anterior solo aplica a /mnt/publico/.
- Aplicamos una regla a todos los archivos cuya extensión termine en .php. Esto es útil para deshabilitar la ejecución de código PHP (como pide el guión de la práctica).
- Forzamos a que cualquier archivo .php se trate como texto plano, en lugar de ejecutarlo como código PHP.
- Añadimos una cabecera HTTP indicando que el contenido debe interpretarse como HTML, requiere que el módulo headers esté habilitado.
- Cerramos el bloque de configuración para archivos .php.
- Si se produce un error 404 (archivo no encontrado), Apache servirá el archivo /errorphp.html en lugar de la página de error por defecto. Esto aplica también si alguien accede a cualquier .php aunque no exista.
- Finalizamos la definición del host virtual. Todo el bloque anterior afecta a conexiones entrantes al puerto 80.

1.1.8 Comprobaciones dentro del contenedor:

Accedemos al contenedor y hacemos una serie de comprobaciones como asegurarnos que el gateway y el dns son los correctos y para comprobar la conectividad haremos varios pings, por ejemplo probamos a hacer ping al dns, que es un contenedor de la subred services y también haremos ping a un contenedor de la red development, por ejemplo al servidor ssh, vemos que ninguno de los pings llegan ya que production esta aislada, aunque esto no quiere decir que no funcione como DNS, el último ping, con destino al servidor Nginx, ubicado en la propia red production si que llega, esto es lo correcto según los guiones de las prácticas anteriores, todo lo descrito se ve en la siguiente imagen:

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ docker exec -it apache_web bash
root@f619edb41097:/# ip route
default via 172.30.0.1 dev eth1
172.18.0.0/16 dev eth0 proto kernel scope link src 172.18.0.6
172.30.0.0/24 dev eth1 proto kernel scope link src 172.30.0.20
root@f619edb41097:/# cat /etc/resolv.conf
nameserver 172.20.0.6
root@f619edb41097:/# ping 172.20.0.6
PING 172.20.0.6 (172.20.0.6) 56(84) bytes of data.
^C
--- 172.20.0.6 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2062ms

root@f619edb41097:/# ping 172.40.0.9
PING 172.40.0.9 (172.40.0.9) 56(84) bytes of data.
^C
--- 172.40.0.9 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2053ms

root@f619edb41097:/# ping 172.30.0.5
PING 172.30.0.5 (172.30.0.5) 56(84) bytes of data.
64 bytes from 172.30.0.5: icmp_seq=1 ttl=64 time=7.62 ms
64 bytes from 172.30.0.5: icmp_seq=2 ttl=64 time=1.45 ms
64 bytes from 172.30.0.5: icmp_seq=3 ttl=64 time=0.948 ms
^C
--- 172.30.0.5 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.948/3.338/7.616/3.031 ms
root@f619edb41097:/#
```

Figura 5: Comprobaciones dentro del contenedor apache_web.

1.1.9 Acceso a Apache desde el host:

Desde el navegador web de nuestro host introducimos lo siguiente en el buscador:

http://localhost:8082

Como podemos observar, se muestra el título de la práctica y el cuerpo de la página tal como se pedía en el enunciado, además, clicando en 'Acceso a la documentación' podremos acceder al directorio 'publico' del servidor FTP previo login.

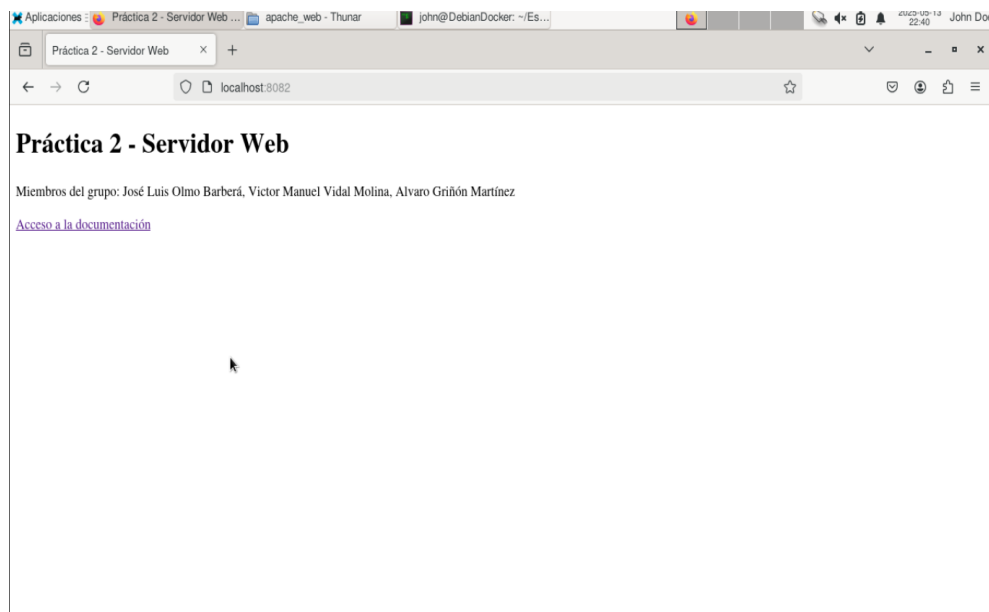


Figura 6: Vista principal del servidor apache_web.

Para hacer login debemos introducir las credenciales indicadas en el enunciado, usuario 'documentos' y contraseña 'GrATiS!', como podemos ver, al introducir las credenciales la web se redirige automáticamente al directorio 'publico' del servidor FTP.

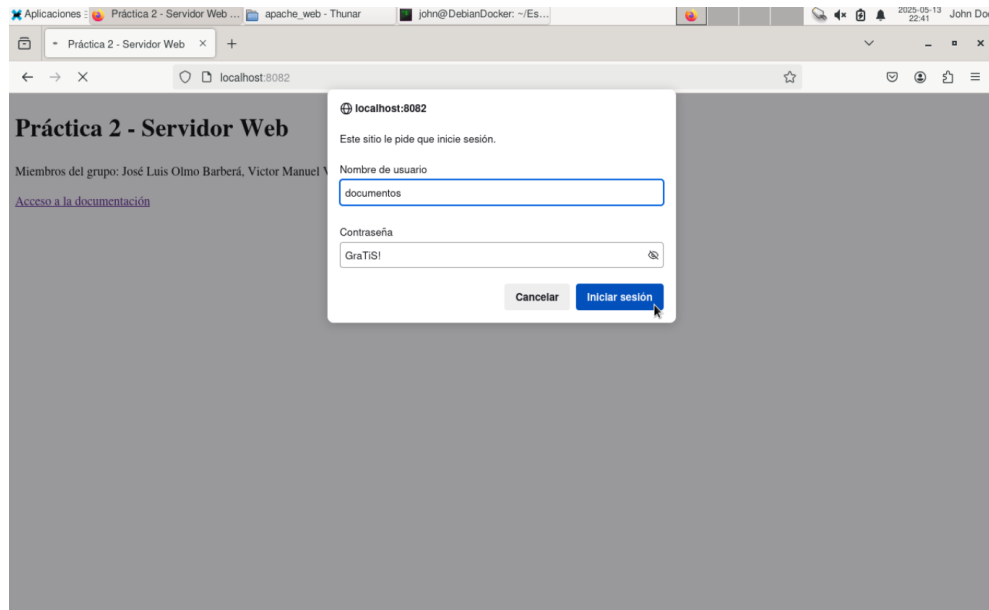


Figura 7: Ventana solicitando login al usuario.

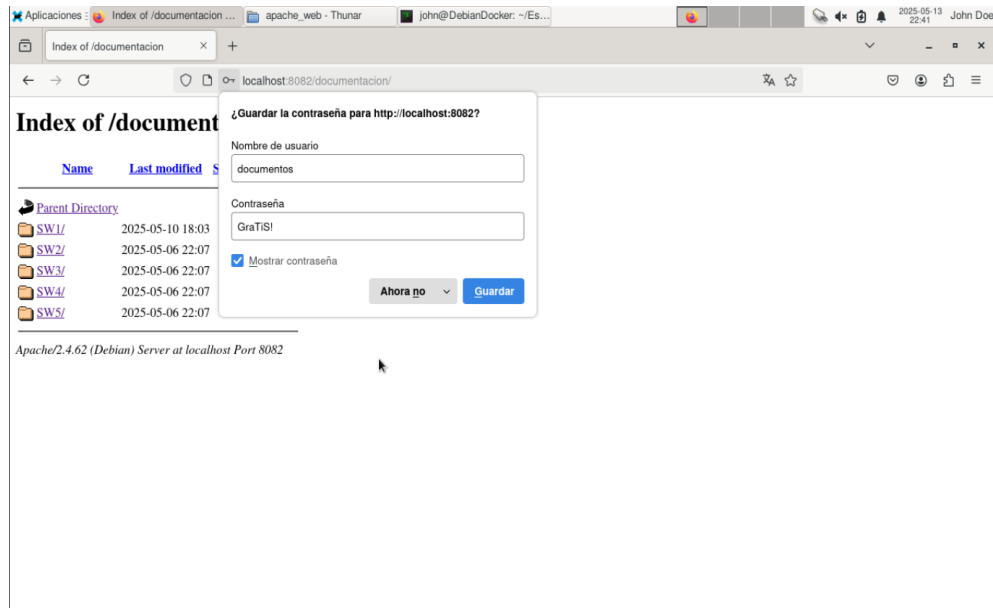


Figura 8: Usuario 'documentos' se ha logueado correctamente.

En este directorio podremos acceder a todos los subdirectorios existentes pero únicamente con permisos de lectura. Accedemos al subdirectorio 'SW1' ya que en la práctica anterior subimos un archivo de prueba. Como se puede observar en las siguientes imágenes, los archivos del directorio están visibles y podemos abrirlos para ver su contenido.

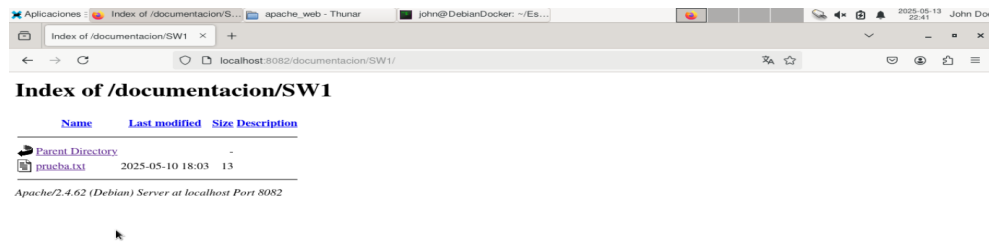


Figura 9: Contenido del subdirectorio 'SW1' del servidor FTP.

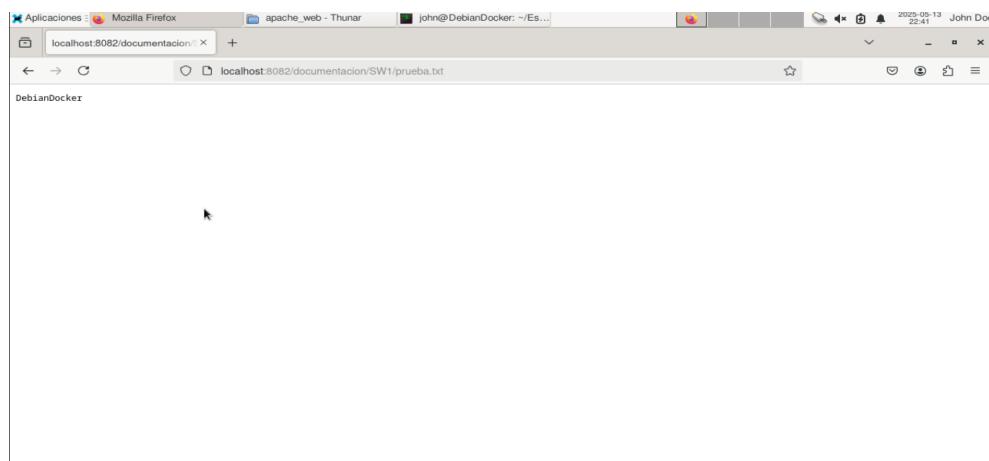


Figura 10: Contenido archivo 'prueba.txt'.

Por último, vamos a comprobar como responde la web ante la petición de cualquier archivo con extensión .php, para ello vamos a introducir en el buscador lo siguiente:

`http://localhost:8082/prueba.php`



Figura 11: Mensaje automático ante peticiones php.

Vemos como la web responde tal y como pide el enunciado, con un mensaje automatizado mediante el archivo 'errorphp.html'.